Centro Federal de Educação Tecnológica de Minas Gerais
Departamento de Computação – Engenharia de Computação – ***Compiladores***
Prof.ª Kecia Marques – 2026/1

Implementação de um Compilador

O trabalho prático a ser realizado na disciplina de Compiladores é a construção de um compilador completo para uma dada linguagem de programação. O trabalho será realizado por etapas, conforme cronograma a seguir. Este documento especifica as características da linguagem e descreve as definições para a realização das etapas do trabalho.

1. Cronograma e Valor

O trabalho vale 30 pontos no total. Ele deverá ser entregue por etapas conforme cronograma abaixo:

Etapa	***Valor***	***Entrega***	***Limite***
1 - Analisador Léxico e Tabela de símbolos	10,0	19/04/2026	24/04/2026
2 - Analisador Sintático	10,0	31/05/2026	05/06/2026
3 - Analisador Semântico e Gerador de Código	10,0	02/07/2026	-

2. Regras

- O trabalho poderá ser realizado individualmente, em dupla ou em trio.
- Não é permitido o uso de ferramentas para geração do analisador léxico e do analisador sintático.
- A implementação deverá ser realizada em C/C++ ou Java. A linguagem utilizada na primeira etapa deverá ser a mesma para as etapas subsequentes. A mudança de linguagem utilizada ao longo do trabalho deverá ser negociada previamente com a professora.
- Realize as modificações necessárias na gramática para a implementação do analisador sintático.
- Não é necessário implementar recuperação de erro, ou seja, ao identificar o erro o compilador pode mostrar a mensagem de erro para o usuário e encerrar o processo de compilação. Entretanto, o relatório deve mostrar todos os erros de compilação do programa analisado.
- A organização do relatório será considerada para fins de avaliação.
- Trabalhos total ou parcialmente iguais receberão avaliação nula.
- Trabalhos realizados com ferramentas de IA receberão avaliação nula.
- Trabalhos total ou parcialmente iguais a projetos apresentados por outros alunos em semestres anteriores receberão avaliação nula (exceto se for o trabalho realizado <u>exclusivamente</u> pelo próprio aluno).

- A tolerância para entrega com atraso é de 1 semana, exceto no caso da Etapa 3 que não será recebida com atraso.
- Os trabalhos somente serão recebidos via SIGAA.

3. Gramática da Linguagem

program ::= **class** identifier "{" [decl-list] body "}"
decl-list ::= decl ";" { decl ";"}
decl ::= type ident-list
ident-list ::= identifier {"," identifier}
type ::= **int** | **string | float**
body ::= "{" stmt-list "}"
stmt-list ::= stmt ";" { stmt ";" }
stmt ::= assign-stmt | if-stmt | do-stmt | repeat-stmt
| read-stmt | write-stmt
assign-stmt ::= identifier ":=" simple_expr
if-stmt ::= **if** "(" condition ")" "{" stmt-list "}"
| **if** "(" condition ")" "{" stmt-list "}" **else** "{" stmt-list "}"
do-stmt ::= **do** "{" stmt-list "}" do-suffix
do-suffix ::= **while** "(" condition ")"
repeat-stmt ::= **repeat** "{" stmt-list "}" stmt-suffix
stmt-suffix ::= **until** "(" condition ")"
read-stmt ::= **read** "(" identifier ")"
write-stmt ::= **write** "(" writable ")"
writable ::= simple-expr
condition ::= expression
expression ::= simple-expr | simple-expr relop simple-expr
simple-expr ::= term | simple-expr addop term
term ::= factor-a | term mulop factor-a
factor-a ::= factor | **not** factor | "-" factor
factor ::= identifier | constant | "(" expression ")"
relop ::= ">" | ">=" | "<" | "<=" | "<>" | "="
addop ::= "+" | "-" | **or**
mulop ::= "*" | "/" | "%" | **and**

Padrão de formação dos tokens

constant → integer_const | literal | real_const
integer_const → nonzero digit* | 0
real_const → interger_const "." digit$^+$
literal → " “ " caractere* " ” "
identifier → letter (letter | digit)*

letter → [A-Za-z]
digit → [0-9]
nonzero → [1-9]
caractere → *um dos 256 caracteres do conjunto ASCII, exceto* as aspas e quebra de linha

4. Outras características da linguagem

- As palavras-chave da linguagem são reservadas.
- Toda variável deve ser declarada antes do seu uso.
- A entrada e a saída da linguagem estão limitadas ao teclado e à tela do computador.
- A linguagem possui comentário de uma linha que começam com "//"
- A linguagem possui comentário de mais de uma linha que começam com "/*" e termina com "*/"
- O operador "+", quando aplicado a dado do tipo string, representa concatenação.
- O operador "%" só se aplica a operandos inteiros e representa o resto da divisão entre dois números.
- Os demais operadores aritméticos são aplicáveis somente aos tipos numéricos.
- O resultado da divisão entre dois números inteiros é um número real.
- Somente tipos iguais são compatíveis nesta linguagem.
- As operações de comparação resultam em valor lógico (verdadeiro ou falso)
- Nos testes (dos comandos condicionais e de repetição) a expressão a ser validada deve ser um valor lógico.
- A semântica dos demais comandos e expressões é a tradicional de linguagens como Java e C.
- A linguagem é *case-sensitive*.
- O compilador da linguagem deverá gerar código a ser executado na máquina VM ou para Jasmin. VM é uma máquina virtual simples, porém possui a limitação de ser executada somente em Windows. O arquivo executável e a documentação de de VM estão disponíveis no Moodle. Jasmin é uma ferramenta que produz bytecodes a serem executados na JVM (Java Virtual Machine).

5. O que entregar

Em cada etapa, deverão ser entregues via Moodle:

- Código fonte do compilador.
- Se desenvolvido em Java, entregar o JAR também.
- Relatório contendo:
    - Forma de uso do compilador

- Descrição da abordagem utilizada na implementação, indicando as principais classes da aplicação e seus respectivos propósitos. Não deve ser incluída a listagem do código fonte no relatório.
- Na etapa 2, as modificações realizadas na gramática
- Resultados dos testes especificados. Os resultados deverão apresentar o programa fonte analisado e a saída do Compilador: reportar sucesso ou reportar o erro e a linha em que ele ocorreu.
    - Na etapa 1, o compilador deverá exibir a sequência de tokens identificados e os símbolos (identificadores e palavras reservadas) instalados na Tabela de Símbolos. Nas etapas seguintes, isso **não** deverá ser exibido.
    - No caso de programa fonte com erro, o relatório deverá mostrar o código fonte analisado e o resultado indicando o erro encontrado. O código fonte deverá ser corrigido para aquele erro, o novo código e o resultado obtido após a correção deverão ser apresentados. Isso deverá ser feito para cada erro que o compilador encontrar no programa fonte.

- Em cada etapa, deverão ser considerados os códigos fontes sem erros da última etapa realizada até então. Por exemplo, na etapa 2, Análise Sintática, os códigos fontes a serem considerados são aqueles sem os possíveis erros léxicos reportados na etapa 1.
- Na etapa 3, o código fonte analisado, o código objeto gerado e o resultado da execução do programa gerado, na VM ou na JVM.

6. Testes

Teste 1

```
class Teste1{
  int base, 1altura;
  float area;


  write("Digite o valor da base:");
  read (base);
  write("Digite o valor da altura:");
  read (altura;
  area :=base*altura/2.0
  write("A área e: " + area);
}
```

Teste 2

```
class Teste2{
/* Teste de comentário
com mais de uma linha

  float a;


  write("Entre com o valor de a: ");
  read (a);

  if (a%2 = 0){
    write (a+ "e par").
  }
  else{
    write (a+ "e impar");
  }



}
```

Teste 3:

```
class MinhaClasse
{
   float a, b, c;

      write("Digite um número");
      read(a);
      write("Digite outro número: ");
      read(b);
      write("Digite mais um número: );
      read(c;

      maior := 0;


      if ( a>b && a>c  )
        maior = a;
      else
         if (b>c)
             maior = b;
         else
             maior = c;


      write("O maior número e: ");
      write(maior);
```

Teste 4:

```
{
// Outro programa de teste

int idade;
string nome, texto;
float bonus, salario;

repeat{
     write("Digite o seu nome: );
     read(nome);
     write("Digite o seu sobrenome");
     read(sobrenome);
     write("Digite a sua idade: ");
     read (idade);
     write("Digite o salario: ");
     read (salrio);
     write("Digite o bonus: ");
     read (bonus);



     if (idade>0 && salario > 0 and bonus >= 0){
          salarioLiquido  =  +salario  -  salário*0.11)  +  (bonus  -
          109.);
          texto = nome + " " + sobrenome + ":";
          write (text);
          write("Salario liquido: ");
          write(salario);
    };
  {until (idade <=0 or Salario <=0 or bonus <0);

}
```

Teste 5

```
class A

/** Verificando fluxo de controle
Programa com if e while aninhados **/

 int i;
 float soma, altura, media, maior:=0, menor:=3;
 int soma;

 {
```

```
  soma := 0;

  write("Quantos dados deseja informar?" );
  read (qtd);

  IF (qtd>=2){
     I:=0;
     do{
       do{
            write("Altura: ");
            read (altura);
       while (altura <= 0 or altura >=2.5);

       soma := soma+altura;
       i := i + 1;
       if (altura > maior){
          maior = altura;
       };
       if (altura < menor){
         menor = Altura;
       };
       }


     }while( i < qtd);

     media := soma / qtd;
     write("Media: " + media);

  }
  else{
    write("Quantidade inválida.");
  }
}
```

Teste 6:

Mostre mais <u>um</u> teste que demonstre o funcionamento de seu compilador. O teste deverá ter comando condicional e de repetição.

****